

Лабораторная работа №8

Реализация панели администратора в WPF и ASP.NET Core

Содержание

1. Цель работы	2
2. Задачи лабораторной работы	2
3. Финальное тестирование	4
4. Ожидаемый результат	4
5. Интерфейс приложения	5
6. Список вопросов	9

1. Цель работы

Освоение расширения функциональности клиент-серверного приложения для поддержки разграничения прав доступа по ролям. Разработка административной панели с возможностью просмотра, фильтрации и удаления данных на основе пользовательской роли.

2. Задачи лабораторной работы

1. Серверная часть (ASP.NET Core)

1.1 Настройка авторизации

- Реализовать механизм JWT-аутентификации.
- Включить поддержку авторизации по ролям с использованием Claims (Customer, Admin).

1.2 Модификация контроллеров

Контроллер `AuthController`:

- Реализовать метод GET `/api/auth/users-with-orders` — получение списка всех пользователей с их заказами (доступен только для роли Admin).
- Реализовать метод DELETE `/api/auth/user/{id}` — удаление пользователя по его идентификатору (доступен только для роли Admin).

Контроллер `OrderController`:

- Обновить метод GET `/api/Order`:
 - Пользователь с ролью Admin получает все заказы;
 - Пользователь с ролью Customer — только собственные (на основе `ClaimTypes.NameIdentifier`).
- Реализовать метод POST `/api/Order`, в котором: значения `UserId` и `CustomerName` подставляются автоматически из токена.
- Реализовать метод GET `/api/Order/user/{id}` — получение заказов конкретного пользователя (только для роли Admin).

1.3 Модификация модели данных и контекста

- Обновить классы сущностей `User`, `Order`, `OrderItem`:
 - Добавить навигационные свойства;
 - Установить аннотации сериализации при необходимости (`[JsonIgnore]`).
- В контексте данных (`StoreDbContext`) настроить связи между сущностями.
- Обеспечить каскадное удаление:

- При удалении пользователя должны автоматически удаляться связанные с ним заказы;
- При удалении заказа — связанные с ним позиции заказа (`OrderItem`).
- Применить изменения к базе данных с использованием миграций Entity Framework Core (`Add-Migration`, `Update-Database`).

2. Клиентская часть (WPF)

2.1 Авторизация и хранение токена

- Реализовать сохранение JWT-токена и роли пользователя в `ApiService`.
- При выполнении HTTP-запросов автоматически добавлять заголовок `Authorization: Bearer <token>`.

2.2 Логика взаимодействия

`LoginViewModel`: после успешной авторизации открывать основное окно приложения (`MainWindow`), передавая в него экземпляр `ApiService`.

`MainViewModel`:

- Ввести свойство `IsAdmin`, определяющее доступность административной панели.
- При наличии административной роли:
 - Загружать список пользователей с заказами (метод `LoadUsersAsync`);
 - Реализовать удаление выбранного пользователя (команда `DeleteUserCommand`);
 - Обновить метод `LoadOrdersAsync`:
 - * Для роли `Admin` — загружать все заказы;
 - * Для роли `Customer` — только собственные.

2.3 Обновление интерфейса пользователя (`MainWindow.xaml`)

Разделить интерфейс на две логические панели:

Панель администратора:

- Список пользователей (`ListBox`);
- Список заказов выбранного пользователя;
- Кнопка удаления пользователя.

Панель клиента:

- Товары;
- Корзина;
- Кнопки создания и отмены заказа;
- Реализовать переключение видимости панелей на основе свойства `IsAdmin`.

3. Финальное тестирование

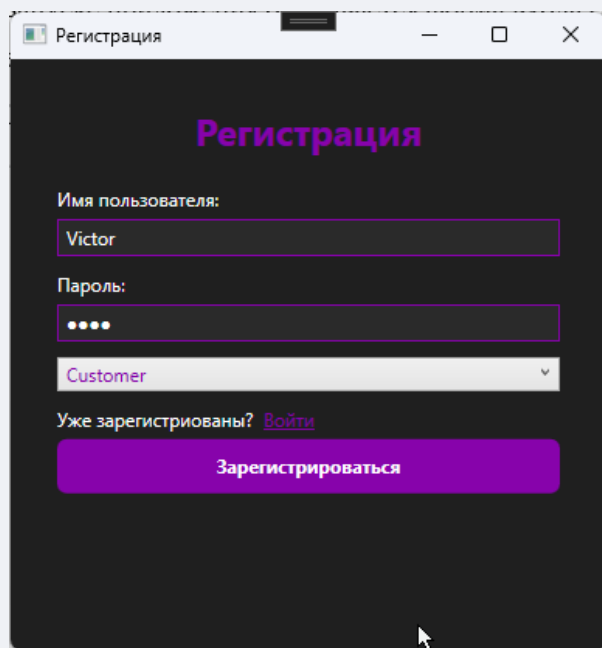
1. Зарегистрировать двух пользователей: одного с ролью Customer, другого — с ролью Admin.
2. Проверить корректность отображения и работы функциональности:
 - Пользователь с ролью Customer должен иметь доступ только к корзине и своим заказам;
 - Пользователь с ролью Admin должен видеть список всех пользователей, иметь возможность просматривать и удалять их, а также видеть соответствующие заказы.
3. Убедиться, что при удалении пользователя его заказы и связанные позиции удаляются автоматически (проверка каскадного удаления).

4. Ожидаемый результат

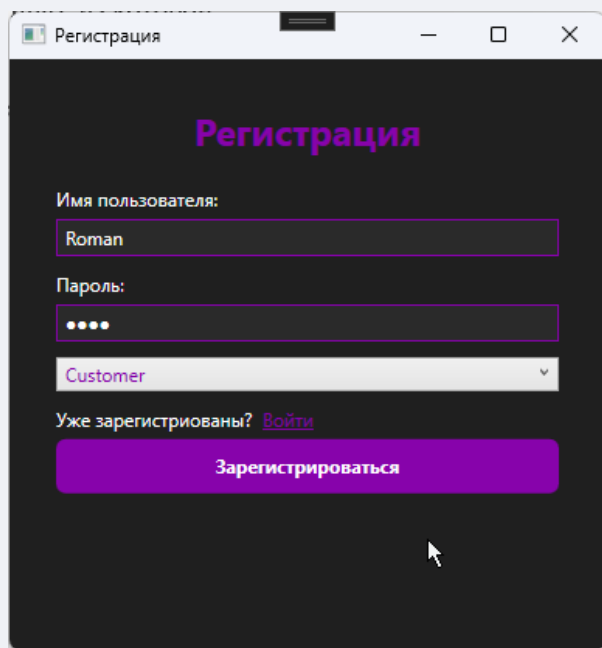
1. Реализован механизм аутентификации и авторизации на основе JWT и ролей.
2. Интерфейс клиента адаптируется в зависимости от роли пользователя.
3. Пользователь с ролью Admin имеет доступ к управлению пользователями и заказами.
4. Обеспечена целостность данных за счёт каскадного удаления в базе данных.

5. Интерфейс приложения

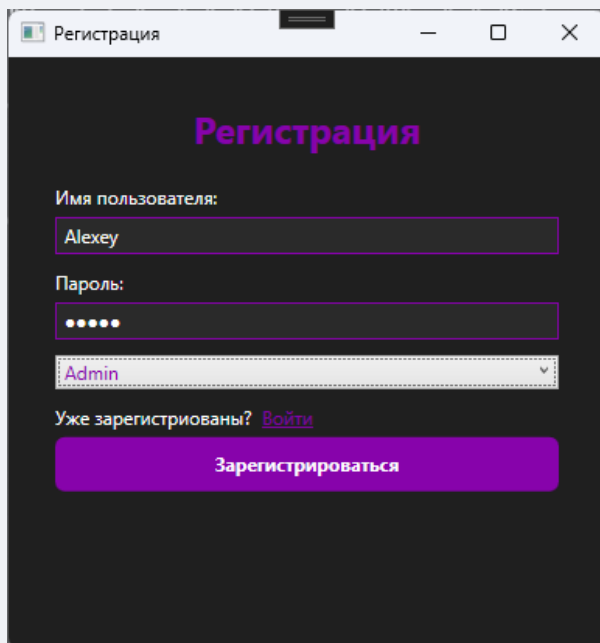
При запуске приложения открывается форма входа (логина), из которой можно перейти на форму регистрации. Необходимо для примера зарегистрировать одного пользователя с ролью «Администратор» и двух пользователей с ролью «Покупатель» (Рисунок 5.1с, Рисунок 5.1а, Рисунок 5.1b).

The screenshot shows a web browser window titled "Регистрация". The form has a title "Регистрация" in red. It contains three input fields: "Имя пользователя:" with the value "Victor", "Пароль:" with masked characters, and a role dropdown menu currently set to "Customer". Below the inputs is a link "Уже зарегистрированы? Войти" and a large red button labeled "Зарегистрироваться".

(а) Регистрация первого покупателя

This screenshot is identical to the previous one, but the "Имя пользователя:" field contains the value "Roman". A mouse cursor is visible near the bottom right of the form.

(b) Регистрация второго покупателя

The screenshot shows the registration form with the title "Регистрация" in red. The "Имя пользователя:" field contains "Alexey", the "Пароль:" field is masked, and the role dropdown menu is set to "Admin". It also includes the "Уже зарегистрированы? Войти" link and the "Зарегистрироваться" button.

(с) Регистрация «Администратора»

Рис. 5.1: Форма регистрации

Далее необходимо войти в приложение в аккаунт администратора и добавить товары. В центре экрана находится список пользователей, зарегистрированных ранее. А в блоке «Заказы» выводятся заказы, оформленные конкретным пользователем (Рисунок 5.2).

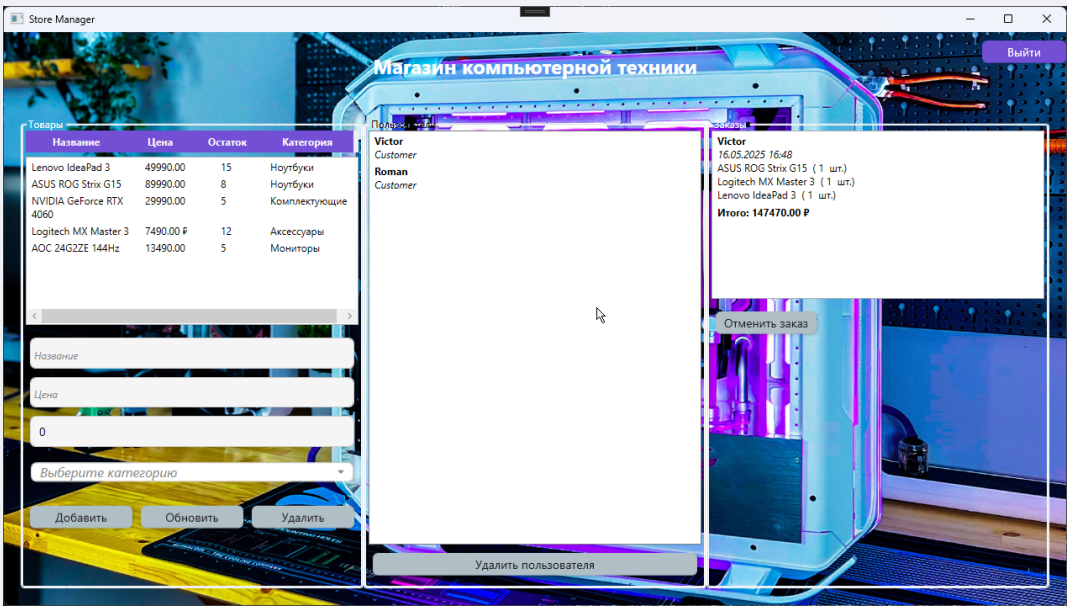


Рис. 5.2: Добавление товаров

Далее необходимо войти под каждым из пользователей и сформировать заказ (Рисунок 5.3, Рисунок 5.4).

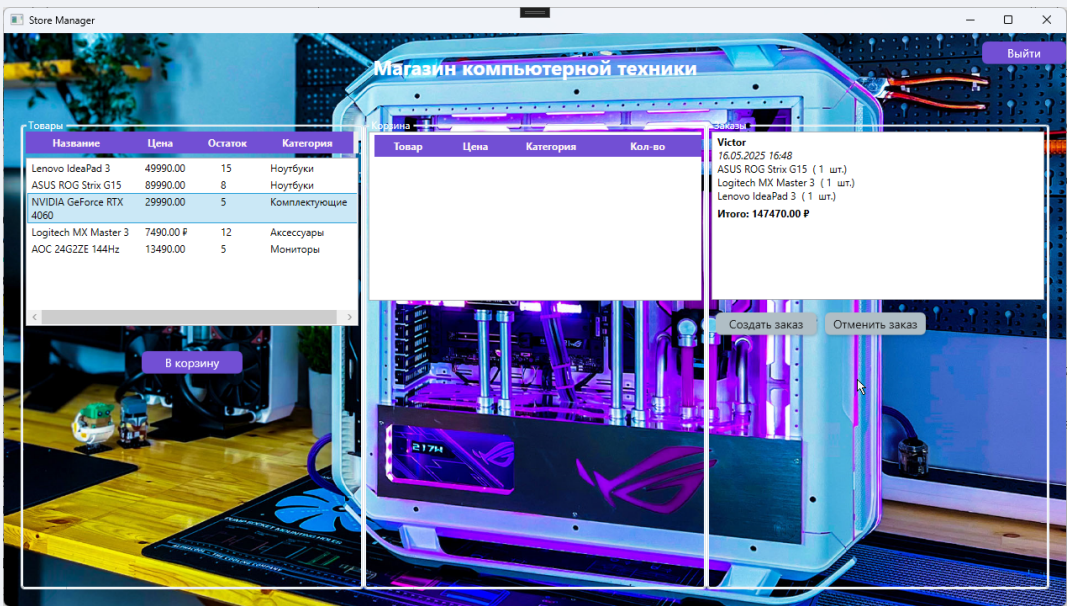


Рис. 5.3: Создание заказов у первого пользователя



Рис. 5.4: Создание заказов у второго пользователя

Затем, выйдя из аккаунта пользователя, необходимо войти в аккаунт администратора и по нажатию на конкретного пользователя просмотреть список его заказов. Также у администратора есть возможность отменить конкретный заказ, созданный пользователем, а также самого пользователя и все созданные им заказы (Рисунок 5.5, Рисунок 5.6).

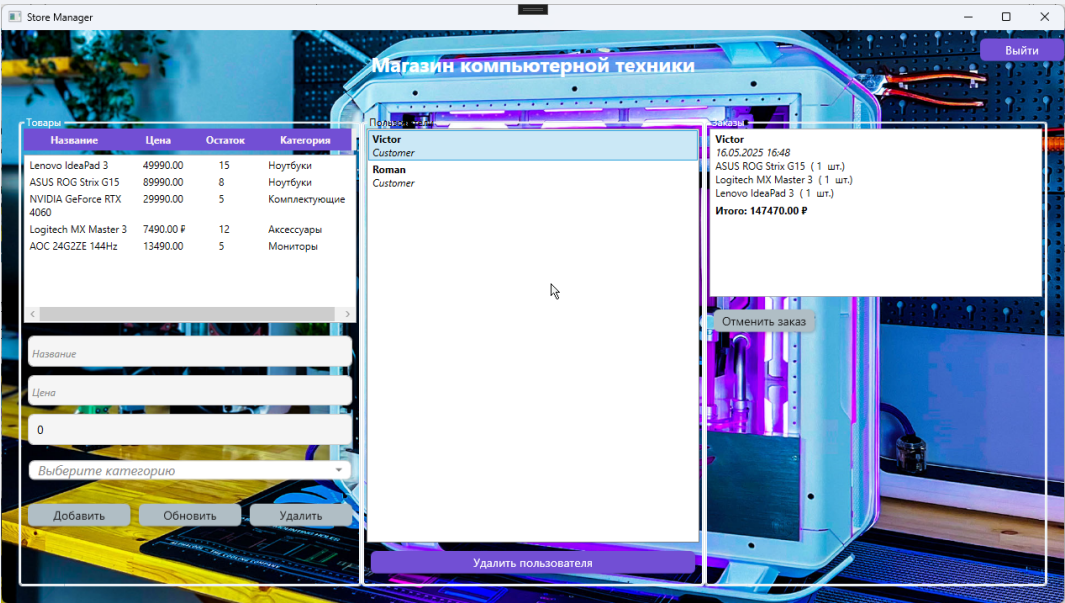


Рис. 5.5: Просмотр заказов пользователя

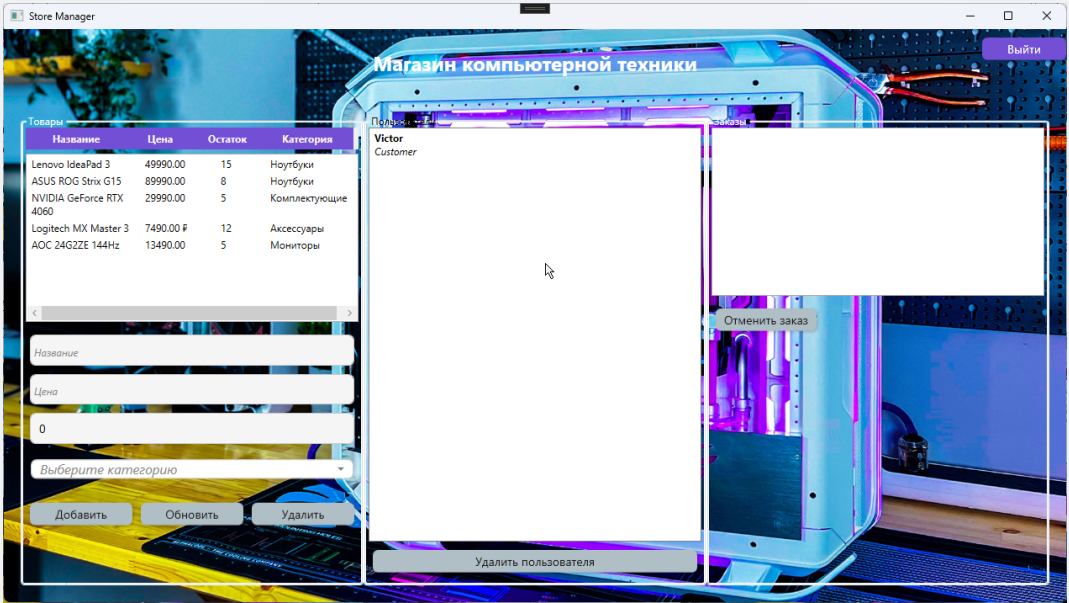


Рис. 5.6: Удаление пользователя

6. Список вопросов

1. Как в ASP.NET Core реализуется ограничение доступа к методам только для пользователей с ролью Admin?
2. Как `OrderController` определяет, какие заказы должен видеть пользователь?
3. Как реализован вывод заказов конкретного пользователя по его `userId`?
4. Почему при создании заказа значения `UserId` и `CustomerName` берутся из токена, а не передаются вручную?
5. Чем отличается поведение интерфейса для администратора и для обычного пользователя?
6. Как реализовано отображение заказов выбранного пользователя на клиенте?
7. Как работает команда удаления пользователя на клиенте?
8. Как настраивается каскадное удаление заказов при удалении пользователя в Entity Framework Core?
9. Почему каскадное удаление может не сработать, даже если оно задано в `DbContext`?
10. Как проверить, что заказы действительно удаляются при удалении пользователя?